

Laboratorio 1

Curador Funcional de Feeds de Reddit

Objetivo

Modelar y resolver problemas utilizando programación funcional en Scala, usando los siguientes conceptos del paradigma funcional y constructores declarativos del lenguaje:

- Principios de inmutabilidad, tipado fuerte y composición de funciones sobre datos estructurados.
- Modelado de dominios mediante la creación de tipos.
- Diferenciar soluciones declarativas de implementaciones imperativas que tengan efectos secundarios y/o dependencia del contexto de computación.
- Procesar colecciones mediante funciones de alto orden como `map`, `filter`, `flatMap` y `fold`, evitando estado mutable.
- Utilizar pattern matching de forma exhaustiva y tipada.
- Emplear `Option` para manejar errores de manera declarativa.

Para poner estos principios en práctica, vamos a construir un sistema en Scala que:

1. Obtenga posts desde **diferentes feeds RSS de Reddit** a partir de una lista de URLs.
2. Filtre y analice el contenido de los posts.
3. Produzca un resumen del filtrado

Restricciones de diseño

Resolver este laboratorio utilizando el **paradigma funcional** implica escribir programas donde el procesamiento de datos se realiza mediante **funciones puras**, evitando el uso de **estado mutable explícito** (asignación destructiva) y **efectos secundarios**. Por ello siempre vamos a preferir usar valores inmutables (`val`) y composiciones de funciones que transforman datos, en lugar de modificar estructuras existentes paso a paso.

Un concepto central que se debe evitar es la **asignación destructiva**. Esta ocurre cuando una variable mutable (`var`) cambia su valor a lo largo del programa, destruyendo su valor

anterior. Este estilo de programación puede generar secciones críticas y condiciones de carrera, ya que introduce estado compartido y dificulta razonar sobre el comportamiento del programa. Aquí vemos un ejemplo de asignación destructiva en Scala, por lo tanto, código NO declarativo, FUERA del paradigma funcional.

```
Java
var total = 0
for (x <- numbers) {
  total = total + x
}
```

En programación funcional se prefiere expresar el mismo cálculo como una **transformación sobre datos**, sin mutación:

```
Java
val total = numbers.sum
```

Otro concepto importante es el de **función de alto orden**. Una función de alto orden es una función que **recibe otras funciones como argumento o devuelve una función como resultado**. Este tipo de funciones permite expresar operaciones generales sobre colecciones de manera declarativa.

Por ejemplo, funciones como `map`, `filter`, `flatMap`, `pipe` y `fold` reciben funciones que indican cómo transformar o combinar los elementos. En el siguiente extracto `map` recibe una función que extrae el título de cada post.

```
None
val titles = posts.map(p => p.title)
```

Tanto en el laboratorio como en el examen se evaluará el uso correcto de estos componentes. La única excepción para utilizar el paradigma imperativo son las funciones de entrada y salida, por ejemplo impresión por pantalla o guardado a archivos.

Se recomienda seguir las buenas prácticas de modelado funcional en Scala:

<https://docs.scala-lang.org/overviews/scala-book/functional-programming.html>

Descripción del problema

Deseamos construir un **filtro funcional de posts** provenientes de múltiples feeds de **Reddit**.

Reddit ofrece una API pública que permite acceder programáticamente a los contenidos publicados en la plataforma. Entre otras opciones, es posible consultar los posts de un subreddit particular utilizando endpoints HTTP que devuelven la información en formato JSON. Por ejemplo, pueden probar abrir la siguiente url en el navegador

<https://www.reddit.com/r/scala/.json?count=10>

Pueden observar que el resultado sigue un formato específico. Este formato incluye datos estructurados sobre cada publicación, como el título, el autor, el puntaje, la fecha de creación y el enlace al contenido. Gracias a esto, aplicaciones externas pueden descargar y procesar automáticamente estos posts para analizarlos, filtrarlos o transformarlos según distintos criterios. En este laboratorio utilizaremos estas capacidades para construir un sistema que procese publicaciones provenientes de distintos feeds de Reddit.

El sistema debe:

1. Leer una lista de suscripciones (subreddits).
2. Descargar los posts de cada feed, transformar y limpiar la información asociada a cada post.
3. Filtrar posts vacíos o irrelevantes.
4. Manejo declarativo de excepciones.
5. Realizar un análisis simple del texto para contar frecuencias de palabras.
6. Obtener estadísticas básicas.

¿Por qué usar el paradigma funcional?

Este problema resulta especialmente adecuado para abordarlo desde el paradigma funcional porque consiste principalmente en procesar y transformar colecciones de datos. Los posts obtenidos desde la API de Reddit pueden modelarse como estructuras inmutables que luego se combinan mediante operaciones declarativas sobre colecciones.

El programa se construye como **una cadena de transformaciones** puras sobre datos, lo que facilita el razonamiento sobre el comportamiento del sistema, mejora la modularidad del código y promueve soluciones más claras y predecibles.

Les recomendamos abordar el problema identificando y separando claramente las distintas funciones de procesamiento que componen la solución (por ejemplo: obtención de datos, parsing, filtrado, transformación y agregación). En el paradigma funcional es conveniente definir funciones pequeñas, puras y bien tipadas, donde cada una realice una única tarea sobre los datos.

Mientras más claramente estén definidas estas funciones y sus tipos de entrada y salida, más fácil será componerlas para construir la solución completa.

Comenzar por aquí

Desde la cátedra les proveemos un esqueleto que deben descargar a partir del siguiente link

<https://drive.google.com/file/d/1ie9txadx8Gu-7opl5N5HMTPv8mD73rW6/view?usp=sharing>

En el mismo se encuentran las instrucciones de instalación y configuración para que puedan comenzar a trabajar. Les recomendamos que al compilar por primera vez estén conectados a una red wifi de banda ancha porque se descargan automáticamente varias librerías.

Su primera tarea es configurar el entorno local y asegurar que compile para cada integrante del equipo. **ASEGURARSE DE INSTALAR SCALA 2.13**

Su segunda tarea es crear un repositorio para el grupo llamado paradigmas26-lab1-gXX donde XX es su número de grupo de dos dígitos, por ejemplo 03.

Su tercera tarea es realizar un commit en el repositorio con **el contenido de la carpeta skeleton** con tag **v0-skeleton** en la rama **main**, teniendo cuidado de no subir ningún objeto binario al repositorio.

Es importante que no copien la carpeta skeleton al repositorio sino que copien su contenido. Si ejecutan el comando **ls** desde la raíz del repositorio deberían ver el archivo [README.md](#)

Entrega

Deberán entregar el código completo del laboratorio, sin binarios, a través del repositorio de GitHub creado.

El commit de entrega debe tener tag **v1**

Fecha de entrega: Jueves 9 de abril 23:59

Ejercicio 1 – Leer la lista de suscripciones

Definir el tipo:

```
Java
type Subscription = (String, String) // (subredditName, url)
```

Se provee un archivo de texto con formato json. Se trata de una lista de suscripciones (diccionarios) con dos claves: `name` es el nombre del subreddit y `url` es donde podemos acceder a los posts.

```
JSON
[
  {
    "name": "Scala",
    "url": "https://www.reddit.com/r/scala/.json?count=10"
  },
  {
    "name": "Learn Programming",
    "url": "https://www.reddit.com/r/learnprogramming/.json?count=10"
  },
  ...
]
```

Requisitos

1. Leer el archivo de suscripciones como `List[Subscription]` a partir de un path relativo
2. No generar resource leaks dejando descriptores de archivos sin cerrar
3. Crear cada `Subscription` usando funciones de alto orden funcionales (por ejemplo: `map`, `filter`, `fold`).

Ejercicio 2 – Descargar los posts

Definir el tipo:

```
Java
type Post = (String, String, String, String) // (subreddit, title,
selftext,)
```

A partir de cada URL que descargaron en el ejercicio 1, ahora tienen que obtener la lista de posts.

1. Descargar el JSON. Utilizaremos la clase `io.Source` como vemos en el siguiente ejemplo. Tener en cuenta que este ejemplo no tiene manejo de errores.

```
Java
import scala.io.Source

val url = "https://www.reddit.com/r/scala.json"
val source = Source.fromURL(url)

val content = source.mkString
source.close()

println(content)
```

2. Extraer los campos:
 - `title`
 - `selftext`
 - `created_utc`
3. Convertir la fecha a un formato estándar (*canonicalización*), que se puede realizar con el siguiente código o similar

```
Java
val createdUtc = (data \ "created_utc").extract[Double].toLong
val date = TextProcessing.formatDateFromUTC(createdUtc)
```

4. Generar una lista de `Post`.

Ejercicio 3 – Filtrar posts vacíos o irrelevantes

Eliminar los posts que:

- No tengan texto (`selftext` vacío).
- Tengan solo espacios.
- No tengan título.

Debe implementarse usando funciones de alto orden.

Ejemplo conceptual (sólo a modo ilustrativo):

Java

```
val numeros = List(1, 2, 3, 4, 5, 6)
val numerosPares = numeros.filter(n => n % 2 == 0)

// Resultado:
// List(2, 4, 6)
```

Ejercicio 4 – Manejo declarativo de excepciones

Si aún no lo hicieron, antes de continuar utilizaremos un menor manejo de errores.

En un enfoque imperativo, los errores suelen manejarse mediante excepciones o bloques `try/catch`. En este modelo, una función declara que retorna cierto tipo de dato, pero en realidad podría interrumpir el flujo de ejecución lanzando una excepción. Esto introduce efectos laterales y hace que el manejo del error dependa de estructuras de control externas.

En cambio, en un enfoque declarativo y funcional, es común representar los posibles errores directamente en el tipo de retorno, por ejemplo usando `Option`. Esto significa que la función declara explícitamente que puede o no producir un resultado.

- Reemplazar o encapsular los bloques `try` con `Option`.
- Manejar explícitamente:
 - fallas de descarga
 - campos faltantes
 - JSON mal formado

Es aceptable utilizar `try/catch` dentro de la implementación de una función cuando es necesario interactuar con bibliotecas externas o APIs imperativas que lanzan excepciones, como la lectura de archivos o el parsing de JSON. Sin embargo, es un requisito que el comportamiento observable de la función sea funcional. Esto significa que la función no debe propagar excepciones como mecanismo principal de manejo de errores. En su lugar, debe expresar explícitamente en su tipo de retorno la posibilidad de fallo, por ejemplo utilizando `Option`.

Ejemplo orientativo:

```
Java
def fetchFeed(url: String): Option[String]
```

Si el feed no se puede descargar, devolver `None`.

En clase se compararán:

- `Option`
- `Try`
- `Either`

Notar que no queremos usar `Try` como mecanismo principal porque no queremos que el sistema falle. `Either` es más flexible que `Option`, pero en este laboratorio trabajaremos con `Option`.

Ejercicio 5 – Contar frecuencias de palabras

Para cada subreddit, contar la frecuencia de las palabras que:

- comiencen con mayúscula
- no sean stopwords

Procesamiento sugerido en dos pasos:

1. Identificar las palabras del texto.
2. Identificar palabras con mayúscula.
3. Contar el número de ocurrencias de cada grupo de palabras equivalentes.

Para este ejercicio, se espera que usen funciones de alto orden que agrupan elementos en Scala, como `groupBy` o `sortBy`.

Lista de stopwords:

```
"the", "about", "above", "after", "again", "against", "all", "am", "an",  
"and", "any", "are", "aren't", "as", "at", "be", "because", "been",  
"before", "being", "below", "between", "both", "but", "by", "can't",  
"cannot", "could", "couldn't", "did", "didn't", "do", "does", "doesn't",  
"doing", "don't", "down", "during", "each", "few", "for", "from", "further",  
"had", "hadn't", "has", "hasn't", "have", "haven't", "having", "he", "he'd",  
"he'll", "he's", "her", "here", "here's", "hers", "herself", "him",  
"himself", "his", "how", "how's", "i", "i'd", "i'll", "i'm", "i've", "if",  
"in", "into", "is", "isn't", "it", "it's", "its", "itself", "let's", "me",  
"more", "most", "mustn't", "my", "myself", "no", "nor", "not", "of", "off",  
"on", "once", "only", "or", "other", "ought", "our", "ours", "ourselves",  
"out", "over", "own", "same", "shan't", "she", "she'd", "she'll", "she's",  
"should", "shouldn't", "so", "some", "such", "than", "that", "that's",  
"the", "their", "theirs", "them", "themselves", "then", "there", "there's",  
"these", "they", "they'd", "they'll", "re", "they've", "this", "those",  
"through", "to", "too", "under", "until", "up", "very", "was", "wasn't",  
"we", "we'd", "we'll", "we're", "we've", "were", "weren't", "what",  
"what's", "when", "when's", "where", "where's", "which", "while", "who",  
"who's", "whom", "why", "why's", "with", "won't", "would",  
"wouldn't", "you", "you'd", "you'll", "you're", "you've", "your", "yours",  
"yourself", "yourselves"
```

Ejercicio 6 - Obtener estadísticas

A partir del conjunto de posts obtenido desde los distintos feeds de Reddit, implementar una operación que calcule el score total acumulado de los posts. El objetivo es recorrer la colección y combinar los valores utilizando un acumulador inmutable. Para ello, también deberán modificar el sistema para leer extraer este campo a partir del json modificando la definición del ejercicio 2.

La solución debe implementarse utilizando la función de alto orden fold (`foldLeft` o `foldRight`). No se considera válida la solución si solo utilizan `map`.

Finalmente, imprimir por pantalla un pequeño informe, en texto plano o Markdown, que reporte la siguiente información para cada suscripción:

1. Nombre y suma total de scores de cada Subscription
2. Palabras más frecuentes con sus ocurrencias, tal como las extrajeron en el ejercicio 5
3. Cinco primeros posts con su título, fecha y URL

Punto estrella ★ – Sistema interactivo

Hacer el sistema interactivo:

1. Mostrar un menú con las suscripciones disponibles y los posts dentro de cada una de ellas.
2. Permitir al usuario seleccionar una suscripción y un post utilizando un menú numérico
3. Mostrar el contenido completo del post seleccionado

Restricciones:

- No romper el diseño funcional.
- Mantener separación entre la lógica de dominio y la lógica de presentación